Contents

Proposal: Scheduled Backup Disk

1. What and why
2. Affected domains
3. Design
4. Blank stick → Backup Disk
5. Console UX
6. Tests
7. Rollout
8. Options considered
9. Settled by Steve (no decision needed)
10. Open questions for Koen

Proposal: Scheduled Backup Disk

Author: Steve (Lead Bot) **Date:** 2026-10-30 **Status:** Draft for Koen. Koen chose Option A on 2026-10-29 11:45 CET. Design Review input from Axle (idea#78 comment 6518163224) and the review chat (Axle, Atlas, Kid, Pixel) is folded in. **Refs:** idea#78. Related: idea#235/#236 (erase guards), #279/#282 (adopt, system-disk refusal), #347 (helper rollout order), `proposals/files-disk.md` (eraseDisk). **Affects:** `agent-engine-dev` (main work), `agent-console-dev`, the App repos (`app-kolibri`, `app-nextcloud`, `app-kiwix`, `app-milkwise`) and `agent-app-dev` conventions, Ops (root helper v6 rollout, hardware tests).

---

1. What and why

A Backup Disk today only backs up when it is plugged in (`immediate`) or when someone asks (`on-demand`). The `scheduled` mode is accepted but never runs. There is no cleanup, so the disk fills up. Every backup stops the App. And there is no clear path from a blank USB stick to a Backup Disk.

This proposal makes a Backup Disk that stays plugged in and:

1. backs up every App instance on the Pi **once a day** at a set local time (default 02:30);
2. **clears old backups automatically when the disk gets full**, always keeping the newest few;
3. backs up Apps **without stopping them** when the App says how to do that safely, and stops them briefly when it doesn't;
4. can be **made from a blank stick in one Console action**.

The first user is MilkWise production on `idea` (wh), which today runs an interim cron copy (see idea#78). That interim job is removed only after this ships, by Atlas or Koen, never with a container restart.

2. Affected domains

Domain	Work
Engine	Scheduler tick, schedule in `BACKUP.yaml`, clock trust, fullness cleanup, per-App live backup with hooks, `makeBackupDisk` op, store summary, capability flag.
Root helper (Ops)	v6 adds `borg-prune`, `borg-delete`, `borg-compact`. Atlas signs off and rolls out before the Engine build.

Domain	Work
Console	Schedule settings on the disk, status per App, grey "time unknown" state, disk-full warning, "Make Backup Disk" action.
App repos	`backup:` block in App metadata (live flag, pre/post hooks). Kolibri, Nextcloud, Kiwix, MilkWise each get theirs.
Ops	Hardware tests on idea04, rollout order, removal of the MilkWise interim job later.

3. Design

3.1 Where the schedule lives

- The schedule belongs to the **Backup Disk**, written in `BACKUP.yaml` on the disk: `schedule: { daily: "02:30" }`. The disk is the source of truth. The Engine the disk is docked to mirrors it into the store (`backupConfig`), as it does for `mode` today.
- Times are **local to the Pi**. A disk moved to another Pi runs at 02:30 on its new Pi.
- Only "daily at HH:MM" in v1. No cron syntax.
- **Only the Engine the disk is docked to runs it** (`dockedTo === localEngine`). No coordination across Engines, so no double runs.
- The scheduler runs **inside the Engine under pm2**, as a one-minute tick next to `timeMonitor` . Never a host cron.
- A scheduled run backs up only instances whose App Disk is docked on the same Engine. Otherwise it records "App Disk not docked" for that instance. No backups over the network in v1.
- New capability `scheduledBackups` , set with the same boot check as `moveAppCrossEngine` . The Console only offers scheduling to Engines that report it.

3.2 When a run is due, and the clock

The Pi has no real-time clock, so after an offline boot the time can be wrong.

- `BACKUP.yaml` keeps `lastRun` (time and Engine uptime at that moment) and `lastAttempt` , `missedRuns` .
- **Clock trust.** The Engine computes `clockTrusted` and `clockReason` . The clock is untrusted when it:
- was never synced since boot (by NTP or by a peer Engine), or
- is earlier than the Engine build timestamp, or
- is earlier than the newest timestamp in the store, or

- is more than a day ahead of a synced peer.
- **Trusted clock:** a run is due when local time has passed today's slot and `lastRun` is before that slot. After a missed day (disk docked late, Engine down) the overdue run happens **once**, not once per missed day.
- **Untrusted clock never stops backups.** The Engine falls back to uptime: one backup per 24 hours of uptime since the last run recorded in `BACKUP.yaml`. A run never depends on the wall-clock date.
- A `lastRun` that lies in the future is treated as stale. A backwards clock jump never causes a second run and never causes a prune.
- `checkedAt` and all result times are set by the Engine.

3.3 Locks and interruptions

- Reuse today's locks: the instance lock plus the Backup Disk lock (`backupLockKeysFor`) and `runningBackupOnDisk`. A scheduled run that finds a lock held **skips and records the skip**. It never queues or waits.
- **Eject and unplug always win.** Eject keeps today's in-flight rules. A USB pull leaves `.backup-in-progress`; on the next dock the Engine retries **once**, then waits for the next slot.
- **The 05:00 pm2 restart:** a run still in progress at 04:55 holds the restart until it ends.
- Anything interrupted is recorded as **INTERRUPTED** and never counts as the newest good backup (its partial borg checkpoint is not used for the "keep newest N" rule).
- An App that was Running before the backup is always Running afterwards, whatever happens.

3.4 Cleanup driven by how full the disk is

Koen's direction: when the disk fills up, old backups are cleared automatically.

- After each successful backup, and before a backup if space looks tight: if free space is **below 15 %**, delete the oldest backups until free space is **above 25 %**, then `borg compact` to give the space back.
- **Always keep the newest 3 good backups per instance**, and never delete an instance's only backup.
- Several instances on one disk: delete round-robin across instances, oldest first, never touching any instance's newest 3.
- Before a backup, estimate the needed space from the last backup's added size. If it doesn't fit even after cleanup, refuse that backup with **BACKUP_DISK_FULL**. The Console shows "Backup Disk full: add a bigger disk". Nothing is ever deleted down to zero.
- Cleanup uses **fullness and backup count only**, never age, so a wrong clock can't delete good backups.
- Cleanup must work at 15 % free (borg needs some working space; tested explicitly).

- **Root helper v6** adds `borg-prune`, `borg-delete` and `borg-compact` to `idea-app-data` (helper v5 today). Engines with an older helper **skip cleanup with HELPER_OUTDATED** and don't fail the backup. Check the borg version on the image (compact needs 1.2+).

3.5 Consistency: back up without stopping, or stop briefly

Koen's concern: copying files while an App writes its database gives a broken copy. The rule is: **a live copy is only taken when the App declares how it is safe; otherwise the App is stopped.**

Each App's metadata gets a `backup:` block:

```
backup:
  method: hook      # none | live | hook | stop (default: stop)
  preHook: "<command run in the App container>"
  postHook: "<command>"   # optional, e.g. delete the dump
  hookTimeout: 300  # seconds
```

- `stop` (default): today's behaviour: stop, borg create, start.
- `live`: copy while running. Only allowed for Apps that write every file via temp file + rename and have no database.
- `hook`: the preHook writes a consistent dump (e.g. SQLite `.backup`, `mysqldump --single-transaction`) into the data folder, borg copies the folder including the dump; restore uses the dump. The live database files are excluded or treated as not authoritative.
- `none`: nothing worth backing up (content that can be re-downloaded).
- Hooks run with `docker exec` **in the App's own container**, as the App's user, with a timeout, while **holding the instance lock throughout**, and the dump is cleaned up afterwards. No root helper change.
- **If a hook fails or times out, the Engine falls back to stop-the-App**, never to a plain live copy. This is recorded as **HOOK_FALLBACK** in the per-App result. `method: none` is also recorded.
- No filesystem snapshots in v1: ext4 on USB has none, and LVM/btrfs would change the disk format and identity paths.
- A night stop/start is safe: all Apps use `pull_policy: never`, so a restart can't pull images.

Per App:

App	Method	How
MilkWise	`live`	Writes every file via temp file + rename, no database. No downtime, which is what production needs.

App	Method	How
Kolibri (0.19.5)	`hook`	Python3 is in the image, so the hook runs SQLite's online backup API inside the container to a dump file. Content channels are copied as files (they don't change while serving).
Nextcloud + MariaDB	`hook`	`mysqldump --single-transaction` in the DB container, with Nextcloud maintenance mode on/off around it. User files are copied live; a file being uploaded at 02:30 may be missed and is picked up the next night. If the dump fails: stop.
Kiwix	`none` for ZIM content (re-loaded from the kit disks), `live` for the small library file.	

Kid defines the metadata contract in `agent-app-dev` and adds the blocks to each App repo, with a Pi test of each hook.

3.6 What the store holds

- Only a **summary**: per App instance the latest result (time, method, result code such as OK, SKIPPED_LOCKED, HOOK_FALLBACK, INTERRUPTED, BACKUP_DISK_FULL, HELPER_OUTDATED) and a short history (last 7), plus disk `freePct`, `clockTrusted`, `clockReason`, `missedRuns`.
- Written **once per run**, never per tick, so the store doesn't grow.
- The full list of backups stays on the disk (borg).

4. Blank stick → Backup Disk

One Engine op, `makeBackupDisk`, with steps and result codes like `moveApp`:

1. **Check:** refuse the system disk and any disk the existing erase guards refuse (idea#235/#236, #282 fix). Show the content summary.
2. **Erase:** reuses the `eraseDisk` path (GPT, one ext4 partition, `META.yaml`, label "IDEA Disk"). Typed confirmation in the Console, as for Erase.
3. **Re-dock:** the Engine waits for the disk to re-appear after formatting and adopts it **exactly once**.
4. **Create:** writes `BACKUP.yaml` with mode `scheduled`, the time (default 02:30), and initialises the borg repo.

An empty IDEA disk or an existing App/Files Disk skips step 2 (combined disks are allowed). Afterwards the disk holds `META.yaml`, `BACKUP.yaml` and a borg repository folder. No new sudoers entry and no new root path: erase uses the existing erase script.

5. Console UX

- Disk card: **Make Backup Disk** (with erase summary and typed confirmation when needed).
- Backup Disk settings: mode (immediate / on-demand / scheduled) and time.
- Status per App: last good backup, method, result. Green when fine, red when overdue or failed, **grey "Backup time unknown: this Pi's clock may be wrong"** when `clockTrusted` is false (instead of a red overdue warning).
- Disk: free space; amber "Cleaning up old backups", red "Backup Disk full".
- "Cleanup paused: Pi needs an update" for HELPER_OUTDATED.

6. Tests

- Unit tests with a fake clock and `skipBorg`: due logic, missed run runs once, uptime fallback, future `lastRun`, backwards jump, lock skip, retry-once, cleanup choice, round-robin, keep-newest-3, disk-full.
- Wrong-clock tests on a Pi use a **test-only Engine time-offset setting** (used by the scheduler and the clock-trust check), never the system clock, because idea04 is on the shared pool store.
- On idea04 with real borg: prune/delete/compact through helper v6; cleanup at 15 % free; USB pull mid-create, mid-prune and mid-compact, then `borg check`; eject during a run before and after the App stop; two instances on one disk; the 04:55 hold on the pm2 restart.
- `makeBackupDisk` on a Pi with an **unplug/replug round trip**; re-dock after formatting happens exactly once; system-disk refusal.
- Per-App hooks: Kolibri, Nextcloud, MilkWise backup while in use, then restore and check the data.
- Pool rules apply (75/65 °C).

7. Rollout

Engine PRs in this order:

1. Schedule, tick, clock trust, missed runs, store summary, capability (no helper change).
2. Helper v6 and fullness cleanup. Helper rollout first: **idea04, then golden idea01, then the pool.** The kits and wh stay on helper v5 and report HELPER_OUTDATED (backups still run, cleanup is skipped).
3. Live backup with App hooks, together with Kid's metadata contract and the App PRs.

4. `makeBackupDisk` and the Console work (Pixel).

MilkWise on wh: when Koen agrees, a Backup Disk is made for it and the live path takes over; the interim cron job is then removed by Atlas per the list in idea#78, without restarting the container.

8. Options considered

- **Host cron (like the interim job):** ruled out. Outside the Engine, invisible to the Console, not tied to the disk.
- **Schedule in the synced store:** every Engine would see it and could double-fire; doesn't work when the disk moves offline. Ruled out.
- **Cron syntax:** more parsing and testing; nobody needs it. Daily only.
- **Age-based retention (keep 30 days):** depends on a correct clock and doesn't protect against a full disk. Koen asked for fullness. Ruled out.
- **Filesystem snapshots (LVM/btrfs):** would change the disk format and identity code. Ruled out for v1.
- **Always live copy:** risks broken database copies (Koen's concern). Ruled out; live only when declared safe.
- **Always stop the App:** not acceptable for MilkWise production. Kept only as default and fallback.
- **Wait for a synced clock before backing up:** a school Pi offline for weeks would never back up. Replaced by the uptime fallback.
- **Blank stick as three separate Console steps:** more room for mistakes; one op with refusable steps reuses the same safe paths.

9. Settled by Steve (no decision needed)

Default time 02:30; cleanup thresholds 15 % / 25 %; keep newest 3; daily only; history of 7 in the store; hook timeout 300 s; retry once after an interrupted run; per-App methods in §3.5; rollout order in §7.

10. Open questions for Koen

1. **Nextcloud:** is a short maintenance-mode window (around a minute at 02:30) acceptable, or must Nextcloud never pause?
2. **MilkWise on wh:** may we make a proper Backup Disk for it (new stick or reuse the interim `MILKWISE-BAK` stick, which would be erased) and then remove the interim cron job?
3. **Kits and wh on helper v5:** OK that they back up without automatic cleanup until a helper update is rolled out there?